# Manual Técnico y Guía de Despliegue

**AngloElectric Project**

**Integrantes del Equipo:** Fiorella Osorio

Cristopher Figueroa

David Valenzuela

Kevin Peña

## 7. Manual Técnico y Guía de Despliegue

### 7.1 Propósito y Alcance

El presente Manual Técnico documenta los requisitos de infraestructura, la organización física del código fuente, las variables de configuración y el procedimiento estandarizado para instalar, inicializar y desplegar la plataforma web Anglo Electric tanto en entornos de desarrollo local como en producción mediante contenedores Docker.

### 7.2 Requisitos Previos del Sistema

Requisitos de Software (Entorno Dockerizado Recomendado):

- Docker Engine: Versión 24.0 o superior para la ejecución de imágenes Linux aisladas.
- Docker Compose: Versión 2.20 o superior para la orquestación multicontenedor a través del archivo docker-compose.yml.
- Git: Cliente de control de versiones para la clonación del repositorio y gestión de ramas.
- Base de Datos Cloud: Instancia activa de PostgreSQL 15+ en Neon DB (Serverless) con cadena de conexión URI y cifrado SSL requerido.

Requisitos para Ejecución Nativa (Desarrollo Local Sin Docker):

- Entorno Frontend: Node.js v20+ (LTS) compatible con Next.js App Router y gestor de paquetes npm.
- Entorno Backend: Python 3.10+, gestor de paquetes pip y soporte de entornos virtuales (venv).

Requisitos Mínimos de Hardware (Servidor AWS EC2 o Máquina Local):

- Procesador (CPU): Mínimo 2 vCPUs para soportar la compilación de Next.js y las peticiones asíncronas concurrentes de Uvicorn (ASGI).
- Memoria RAM: 4 GB RAM (mínimo 2 GB libres durante el empaquetado de producción npm run build).
- Almacenamiento: 10 GB de espacio libre en disco para imágenes base de Docker (python:3.10-slim y node:20-alpine), paquetes de node_modules y librerías científicas de Python (scikit-learn, pandas, numpy, joblib).

### 7.3 Estructura Física del Repositorio

El proyecto está organizado bajo una arquitectura monorepo desacoplada en dos servicios principales (frontend y backend), orquestados desde el directorio raíz:

```
Capstone-feature-mejoras-capstone/
├── docker-compose.yml          # Orquestador de contenedores (Frontend + Backend)
├── README.md                   # Documentación general del repositorio
├── requirements.txt            # Dependencias globales de referencia
├── backend/                    # Servicio API REST y Motor IA (FastAPI)
│   ├── core/
│   │   ├── __init__.py
│   │   ├── config.py           # Carga y validación de variables de entorno
│   │   ├── database.py         # Configuración de sesión SQLAlchemy hacia Neon DB
│   │   └── security.py         # Cifrado Bcrypt y validación de tokens JWT
│   ├── routers/                # Controladores de endpoints RESTful por dominio
│   │   ├── auth.py             # Inicio de sesión, registro y emisión de JWT
│   │   ├── usuarios.py         # Gestión de perfiles y roles (RBAC)
│   │   ├── categorias.py       # Clasificación de equipamiento solar
│   │   ├── productos.py        # CRUD de catálogo y filtrado de kits (es_kit)
│   │   ├── cotizaciones.py     # Registro y consulta de simulaciones
│   │   ├── ia.py               # Endpoint de inferencia predictiva KNN
│   │   ├── pedidos.py          # Procesamiento de órdenes de compra
│   │   ├── pagos.py            # Integración de pasarela y validación de pago
│   │   └── agenda.py           # Programación de visitas e instalaciones técnicas
│   ├── main.py                 # Punto de entrada de FastAPI y configuración CORS
│   ├── models.py               # Modelos ORM relacionales (8 tablas principales)
│   ├── models_extra.py         # Extensiones modulares del modelo de datos
│   ├── schemas.py              # Esquemas Pydantic de validación de entrada/salida
│   ├── modelo_knn.py           # Lógica de preprocesamiento e inferencia ML
│   ├── modelo_solar.joblib     # Modelo K-Nearest Neighbors entrenado y serializado
│   ├── modelo_solar.json       # Metadatos y parámetros de soporte del modelo
│   ├── inicializar.py          # Script de seeding inicial (roles, admin y catálogo)
│   ├── probar_modelo.py        # Script de pruebas unitarias para inferencia KNN
│   └── requirements.txt        # Dependencias del servidor Python
└── frontend/                   # Aplicación Web Cliente (Next.js App Router)
    ├── public/                 # Activos estáticos (imágenes de paneles, kits, logos)
    ├── src/
    │   ├── app/
    │   │   ├── (admin)/admin/  # Rutas protegidas (Dashboard, CRUD, Cotizaciones)
    │   │   └── (tienda)/       # Rutas cliente (Catálogo, Cotizador IA, Checkout, Pago, Agenda)
    │   ├── components/         # Componentes UI (CotizadorIA, ResultadoKNN, ChatbotSolar,
CartSidebar)
    │   ├── context/            # Estado global (CartContext, AlertContext)
    │   ├── hooks/              # Hooks personalizados (useAuthForm)
    │   ├── providers/          # Proveedores de contexto (AppProviders)
```

```
│   ├── services/api.js        # Cliente HTTP para comunicación con FastAPI
│   └── utils/                 # Formateadores CLP, validaciones y comunas (chileData.js)
├── package.json               # Dependencias de React, Next.js y Tailwind CSS
└── next.config.mjs            # Configuración de compilación de Next.js
```

### 7.4 Configuración de Variables de Entorno (.env)

El ecosistema depende de configuraciones inyectadas mediante variables de entorno para proteger las credenciales sensibles fuera del código fuente. Antes de iniciar los servicios, se debe crear un archivo .env en el directorio raíz con las siguientes llaves:

| Variable | Servicio | Tipo / Restricción | Descripción Técnica (Sin Valores Reales) |
|---|---|---|---|
| **DATABASE_URL=** | Backend | String (URI SSL) | Cadena de conexión hacia la instancia PostgreSQL en Neon DB (sin exponer usuario, contraseña ni host real). |
| **SECRET_KEY=** | Backend | String (Secreto) | Llave criptográfica privada de alta entropía para la firma de tokens JWT (generada localmente por el administrador). |
| **ALGORITHM=** | Backend | String | Identificador del algoritmo de cifrado simétrico utilizado por el módulo de seguridad para la firma JWT. |
| **ACCESS_TOKEN_EXPIRE_MINUTES=** | Backend | Integer | Valor numérico entero que define el tiempo máximo de vigencia de las sesiones en minutos. |
| **NEXT_PUBLIC_API_URL=** | Frontend | String (URL) | Dirección base del servicio Backend API REST para el consumo desde el contenedor de interfaz Next.js. |

### 7.5 Instrucciones de Instalación y Despliegue Paso a Paso

Opción A: Despliegue Automatizado con Docker Compose (Producción / QA)

- Paso 1 — Clonar el repositorio y posicionarse en la rama del proyecto:

```
git clone https://github.com/Palta2639/Capstone.git
cd Capstone
git checkout feature-mejoras-capstone
```

- Paso 2 — Configurar el archivo .env: Crear el archivo .env en la raíz del proyecto definiendo las variables detalladas en la sección 7.4.
- Paso 3 — Construir y levantar los contenedores aislados:

```
docker-compose up --build -d
```

- Paso 4 — Inicializar la base de datos y poblar datos semilla (Seeding): Ejecutar el script de inicialización para verificar la estructura de las 8 tablas en Neon DB y cargar roles, usuario administrador y catálogo base:

```
docker exec -it backend_service python inicializar.py
```

- Paso 5 — Verificar los servicios expuestos:
- Interfaz Gráfica Web (Next.js): http://localhost:3000
- Documentación Interactiva API REST (Swagger UI): http://localhost:8000/docs

Opción B: Ejecución Nativa para Desarrollo Local

- 1. Iniciar el Servidor Backend (FastAPI + Modelo KNN):

```
cd backend
python -m venv venv
# En Windows: venv\Scripts\activate | En Linux/Mac: source venv/bin/activate
pip install -r requirements.txt
python inicializar.py
python probar_modelo.py
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

- 2. Iniciar la Aplicación Frontend (Next.js):

```
cd frontend
npm install
npm run dev
```

### 7.6 Guía de Solución de Problemas Comunes

La siguiente tabla resume las incidencias técnicas más frecuentes durante la puesta en marcha y su resolución:

| Problema / Error Detectado | Causa Probable | Solución Técnica |
|---|---|---|
| sqlalchemy.exc.OperationalError: SSL SYSCALL error | Neon DB cierra conexiones inactivas en su capa serverless o falta el parámetro SSL en la URI. | Verificar que DATABASE_URL incluya ?sslmode=require al final y que el motor SQLAlchemy en database.py tenga configurado pool_pre_ping=True. |
| Error CORS policy: No 'Access-Control-Allow-Origin' | El origen desde el que realiza peticiones Next.js no está incluido en la lista blanca del backend. | Revisar CORSMiddleware en backend/main.py y confirmar que http://localhost:3000 esté declarado en allow_origins. |

| | | |
|---|---|---|
| FileNotFoundError: modelo_solar.joblib al cotizar | El directorio de trabajo desde donde se invocó Uvicorn no coincide con la ruta relativa del binario. | Ejecutar Uvicorn situándose dentro de la carpeta /backend o resolver la ruta absoluta mediante os.path.dirname(__file__) en modelo_knn.py. |
| Error 401 Unauthorized en Cotizador IA o Checkout | El token JWT expiró o no se adjuntó en la cabecera Authorization: Bearer <token>. | Cerrar sesión en la interfaz web y volver a autenticarse mediante AuthModal para regenerar un token vigente. |
| Fallo de paquetes en node_modules al levantar Docker | Sobrescritura de binarios compilados en el sistema anfitrión (Windows) sobre el contenedor Linux Alpine. | Ejecutar docker-compose down -v y reconstruir con docker-compose up --build para regenerar los volúmenes anónimos limpios. |